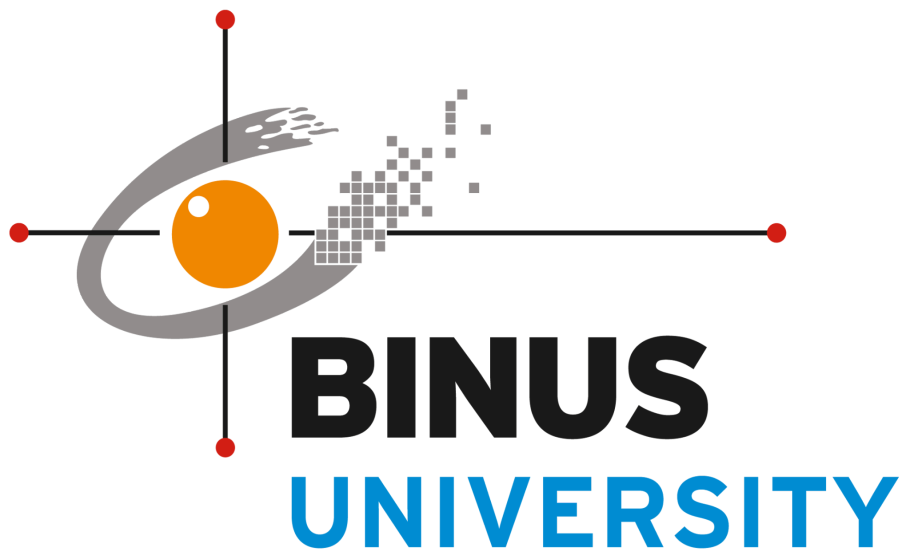


Batik Pattern Classification



Final Project Deep Learning

Anggota Kelompok :

- Vincent - 2702279325
- Vincent Antonio Ekachandra - 2702240891
- Evan Fanuel - 2702327116

I. Latar Belakang

Batik merupakan warisan budaya Indonesia yang memiliki nilai seni dan filosofi tinggi, dengan beragam motif yang mencerminkan identitas daerah, sejarah, serta makna simbolik tertentu. Seiring berkembangnya industri kreatif dan digitalisasi, kebutuhan akan sistem otomatis untuk mengenali dan mengklasifikasikan motif batik semakin meningkat. Proses identifikasi batik secara manual membutuhkan keahlian khusus dan waktu yang tidak sedikit, sehingga berpotensi menimbulkan kesalahan subjektif, terutama ketika jumlah data dan variasi motif semakin besar.

Perkembangan teknologi *deep learning*, khususnya *Convolutional Neural Network* (CNN), telah menunjukkan performa yang sangat baik dalam pengolahan citra digital. Salah satu arsitektur CNN yang populer adalah **ResNet50**, yang mampu mengatasi permasalahan *vanishing gradient* melalui penggunaan *residual connections*. Dengan memanfaatkan teknik **transfer learning** dan **fine-tuning**, model ResNet50 yang telah dilatih pada dataset berskala besar seperti ImageNet dapat diadaptasi untuk klasifikasi motif batik dengan jumlah data yang relatif terbatas, namun tetap menghasilkan akurasi yang tinggi.

Agar hasil penelitian dapat dimanfaatkan secara praktis, diperlukan sebuah aplikasi yang mudah digunakan oleh pengguna umum. **Streamlit** dipilih sebagai media *deployment* karena mampu menyajikan antarmuka web yang interaktif, ringan, dan cepat dalam mengintegrasikan model *machine learning*. Dengan adanya aplikasi berbasis Streamlit, pengguna dapat mengunggah citra batik dan memperoleh hasil klasifikasi secara langsung. Diharapkan sistem ini dapat membantu pelestarian budaya batik, mendukung edukasi masyarakat, serta menjadi solusi inovatif dalam pengenalan motif batik berbasis teknologi kecerdasan buatan.

II. Dataset

Dataset yang digunakan dalam proyek ini merupakan dataset citra batik yang telah terstruktur dengan baik ke dalam dua subset utama, yaitu **data latih (train)** dan **data uji (test)**. Setiap subset dibagi lagi ke dalam beberapa folder kelas yang merepresentasikan motif batik tertentu, misalnya *Aceh_Pintu_Aceh* dan motif lainnya. Struktur folder ini mengikuti standar input *image classification* pada Keras, sehingga memudahkan proses pemuatan data secara otomatis. Pembagian dataset ke dalam train dan test menunjukkan bahwa proyek ini telah memperhatikan prinsip evaluasi model yang baik, yaitu menguji performa model pada data yang tidak pernah dilihat sebelumnya. Keberadaan citra batik dalam berbagai variasi visual juga membantu model dalam mempelajari karakteristik motif secara lebih general.

Preview Dataset :



Dataset Handling :

```
IMG_SIZE = (224, 224)
BATCH_SIZE = 32
EPOCHS_WARMUP = 15
EPOCHS_FINETUNE = 30
SEED = 42
```

```
TRAIN_DIR = "../batik_dataset/train/"
TEST_DIR = "../batik_dataset/test/"
```

```
train_ds = image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=0.2,
    subset="training",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical",
)

val_ds = image_dataset_from_directory(
    TRAIN_DIR,
    validation_split=0.2,
    subset="validation",
    seed=SEED,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical",
)

test_ds = image_dataset_from_directory(
    TEST_DIR,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical",
    shuffle=False
)

class_names = train_ds.class_names
NUM_CLASSES = len(class_names)
```

```
def preprocess(image, label):
    image = preprocess_input(image)
    return image, label

AUTOTUNE = tf.data.AUTOTUNE
train_ds = (train_ds.map(lambda x, y: (data_augmentation(x), y)).map(preprocess).prefetch(AUTOTUNE))
val_ds = val_ds.map(preprocess).prefetch(AUTOTUNE)
test_ds = test_ds.map(preprocess).prefetch(AUTOTUNE)
```

Preprocessing :

```
data_augmentation = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.2),
    layers.RandomZoom(0.2),
    layers.RandomContrast(0.2),
])
```

III. Model

Model yang digunakan dalam proyek ini adalah ResNet50 dengan pendekatan transfer learning dan fine-tuning, yang disimpan dalam file `batik_classification_model.keras`. ResNet50 dipilih karena kemampuannya dalam mengekstraksi fitur visual yang kompleks melalui arsitektur deep residual network. Dengan memanfaatkan bobot awal dari dataset ImageNet, model dapat mengenali pola dasar seperti tekstur, bentuk, dan warna batik secara lebih efektif meskipun jumlah dataset terbatas. Proses fine-tuning dilakukan dengan membuka beberapa layer terakhir untuk dilatih ulang agar model lebih adaptif terhadap karakteristik khusus motif batik. Informasi kelas disimpan dalam file `class_names.npy`, yang berfungsi sebagai pemetaan antara output model dan label kelas.

Arsitektur ResNet50 didasarkan pada konsep Deep Residual Learning. Berbeda dengan Convolutional Neural Network (CNN) konvensional yang bertujuan mempelajari fungsi pemetaan target $H(x)$ secara langsung, ResNet memformulasi ulang lapisan-lapisannya untuk mempelajari fungsi residual $F(x)$

$$y = \sigma(F(x, \{W_i\}) + x)$$

Perbedaan fundamental antara ResNet50 dan CNN konvensional (seperti VGGNet) terletak pada mekanisme propagasi sinyal dan efisiensi arsitektur. Sementara CNN konvensional menggunakan aliran feed-forward sekuensial yang rentan terhadap masalah hilangnya gradien saat jaringan semakin dalam, ResNet50 mengatasinya dengan mengimplementasikan Skip Connection. Jalur pintas ini menyediakan lintasan langsung bagi gradien untuk mengalir tanpa hambatan berarti selama backpropagation, sebuah mekanisme krusial yang memitigasi masalah penyusutan gradien sekaligus mencegah degradasi akurasi pada jaringan dalam

ResNet50 Modelling :

```
base_model = ResNet50(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)

base_model.trainable = False # Freeze backbone
```

```
inputs = layers.Input(shape=(224, 224, 3))
x = data_augmentation(inputs)
x = tf.keras.applications.resnet50.preprocess_input(x)

x = base_model(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.BatchNormalization()(x)
x = layers.Dense(256, activation="relu")(x)
x = layers.Dropout(0.5)(x)

outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = models.Model(inputs, outputs)
```

```
model.compile(
    optimizer=Adam(learning_rate=1e-4)
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()
```

Model Architecture :

Layer (type)	Output Shape	Param #	Connected to	conv2_block2_1_conv (Conv2D)	(None, 56, 56, 64)	16,640	conv2_block1_out_ (Conv2D)	conv3_block1_1_bn (BatchNormalization)	(None, 56, 56, 512)	conv3_block1_1_c_ (Conv2D)
input_layer_2 (InputLayer)	(None, 224, 224, 3)	0	-	conv2_block2_1_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block2_1_c_ (Conv2D)	conv3_block1_1_relu (Activation)	(None, 56, 56, 512)	conv3_block1_1_b_ (BatchNormalization)
conv1_pad (ZeroPadding2D)	(None, 224, 224, 3)	0	input_layer_2[1:3]	conv2_block2_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block2_1_b_ (Conv2D)	conv3_block1_2_conv (Conv2D)	(None, 56, 56, 161,568)	conv3_block1_2_r_ (Conv2D)
conv1_conv (Conv2D)	(None, 112, 112, 64)	8,448	conv1_pad[1:3]	conv2_block2_2_conv (Conv2D)	(None, 56, 56, 64)	16,640	conv2_block2_2_r_ (Conv2D)	conv3_block1_2_bn (BatchNormalization)	(None, 56, 56, 512)	conv3_block1_2_c_ (Conv2D)
conv1_bn (BatchNormalization)	(None, 112, 112, 64)	256	conv1_conv[1:3]	conv2_block2_2_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block2_2_c_ (Conv2D)	conv3_block1_2_relu (Activation)	(None, 56, 56, 512)	conv3_block1_2_b_ (BatchNormalization)
conv1_relu (Activation)	(None, 112, 112, 64)	0	conv1_bn[1:3]	conv2_block2_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block2_2_b_ (Conv2D)	conv3_block1_3_conv (Conv2D)	(None, 56, 56, 161,568)	conv2_block1_out_ (Conv2D)
pool1_pad (ZeroPadding2D)	(None, 112, 112, 64)	0	conv1_relu[1:3]	conv2_block2_3_conv (Conv2D)	(None, 56, 56, 128)	16,640	conv2_block2_3_r_ (Conv2D)	conv3_block1_3_bn (BatchNormalization)	(None, 56, 56, 512)	conv3_block1_3_c_ (Conv2D)
pool1_pool (MaxPooling2D)	(None, 56, 56, 64)	0	pool1_pad[1:3]	conv2_block2_3_bn (BatchNormalization)	(None, 56, 56, 128)	1,024	conv2_block2_3_c_ (Conv2D)	conv3_block1_3_relu (Activation)	(None, 56, 56, 512)	conv3_block1_3_b_ (BatchNormalization)
conv2_block1_1_conv (Conv2D)	(None, 56, 56, 64)	4,160	pool1_pool[1:3]	conv2_block2_add (Add)	(None, 56, 56, 128)	0	conv2_block2_out_ (Conv2D)	conv3_block1_3_bn (BatchNormalization)	(None, 56, 56, 512)	conv3_block1_3_c_ (Conv2D)
conv2_block1_1_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block1_1_c_ (Conv2D)	conv2_block2_out_ (Conv2D)	(None, 56, 56, 128)	0	conv2_block2_add_ (Add)	conv3_block1_add (Add)	(None, 56, 56, 512)	conv3_block1_8_b_ (BatchNormalization)
conv2_block1_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_1_b_ (BatchNormalization)	conv2_block3_1_conv (Conv2D)	(None, 56, 56, 128)	16,640	conv2_block2_out_ (Conv2D)	conv3_block1_out_ (Conv2D)	(None, 56, 56, 512)	conv3_block1_add_ (Add)
conv2_block1_2_conv (Conv2D)	(None, 56, 56, 64)	16,640	conv2_block1_1_r_ (Conv2D)	conv2_block3_1_bn (BatchNormalization)	(None, 56, 56, 128)	256	conv2_block3_1_c_ (Conv2D)	conv3_block2_1_conv (Conv2D)	(None, 56, 56, 65,536)	conv3_block1_out_ (Conv2D)
conv2_block1_2_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block1_2_c_ (Conv2D)	conv2_block3_1_relu (Activation)	(None, 56, 56, 128)	0	conv2_block3_1_b_ (BatchNormalization)	conv3_block2_1_bn (BatchNormalization)	(None, 56, 56, 512)	conv3_block2_2_c_ (Conv2D)
conv2_block1_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_2_b_ (BatchNormalization)	conv2_block3_2_conv (Conv2D)	(None, 56, 56, 128)	16,640	conv2_block3_2_r_ (Conv2D)	conv3_block2_1_relu (Activation)	(None, 56, 56, 512)	conv3_block2_2_b_ (BatchNormalization)
conv2_block1_3_conv (Conv2D)	(None, 56, 56, 64)	16,640	pool1_pool[1:3]	conv2_block3_2_bn (BatchNormalization)	(None, 56, 56, 128)	256	conv2_block3_2_c_ (Conv2D)	conv3_block2_2_conv (Conv2D)	(None, 56, 56, 161,568)	conv3_block2_2_r_ (Conv2D)
conv2_block1_3_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block1_3_c_ (Conv2D)	conv2_block3_2_relu (Activation)	(None, 56, 56, 128)	0	conv2_block3_2_b_ (BatchNormalization)	conv3_block2_2_bn (BatchNormalization)	(None, 56, 56, 512)	conv3_block2_2_c_ (Conv2D)
conv2_block1_3_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_3_b_ (BatchNormalization)	conv2_block3_out_ (Conv2D)	(None, 56, 56, 128)	0	conv2_block3_add_ (Add)	conv3_block2_2_relu (Activation)	(None, 56, 56, 512)	conv3_block2_2_b_ (BatchNormalization)
conv2_block1_4_conv (Conv2D)	(None, 56, 56, 64)	16,640	conv2_block1_3_r_ (Conv2D)	conv2_block3_3_conv (Conv2D)	(None, 56, 56, 128)	16,640	conv2_block3_3_r_ (Conv2D)	conv3_block2_2_bn (BatchNormalization)	(None, 56, 56, 512)	conv3_block2_2_c_ (Conv2D)
conv2_block1_4_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block1_4_c_ (Conv2D)	conv2_block3_3_bn (BatchNormalization)	(None, 56, 56, 128)	1,024	conv2_block3_3_c_ (Conv2D)	conv3_block2_2_relu (Activation)	(None, 56, 56, 512)	conv3_block2_2_b_ (BatchNormalization)
conv2_block1_4_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_4_b_ (BatchNormalization)	conv2_block3_3_relu (Activation)	(None, 56, 56, 128)	0	conv2_block3_out_ (Conv2D)	conv3_block2_2_conv (Conv2D)	(None, 56, 56, 65,536)	conv3_block2_2_r_ (Conv2D)
conv2_block1_add (Add)	(None, 56, 56, 64)	0	conv2_block1_4_b_ (BatchNormalization)	conv2_block3_out_ (Conv2D)	(None, 56, 56, 128)	0	conv2_block3_add_ (Add)	conv3_block2_2_bn (BatchNormalization)	(None, 56, 56, 512)	conv3_block2_2_c_ (Conv2D)
conv2_block1_out_ (Conv2D)	(None, 56, 56, 64)	0	conv2_block1_add_ (Add)	conv3_block1_1_conv (Conv2D)	(None, 56, 56, 128)	16,640	conv2_block3_out_ (Conv2D)	conv3_block2_2_relu (Activation)	(None, 56, 56, 512)	conv3_block2_2_b_ (BatchNormalization)

[illegible]

```
early_stopping = tf.keras.callbacks.EarlyStopping(  
    monitor="val_loss",  
    patience=5,  
    restore_best_weights=True  
)  
  
history_warmup = model.fit(  
    train_ds,  
    validation_data=val_ds,  
    epochs=EPOCHS_WARMUP,  
    callbacks=[early_stopping]  
)
```

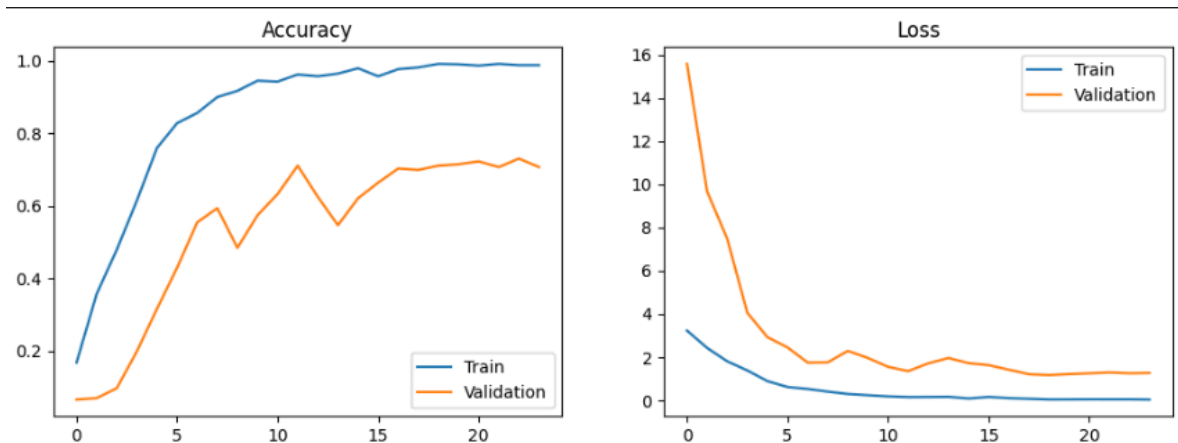
Fine Tuned-Training :

```
# fine-tuning
for layer in model.layers:
    layer.trainable = True

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

fine_tune_history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=EPOCHS_FINETUNE,
    callbacks=[early_stopping]
)
```

Graphs :



IV. Evaluation/Test

Evaluasi model dilakukan dengan menggunakan data uji (test) yang terpisah dari data latih, sehingga hasil pengujian mencerminkan kemampuan generalisasi model. Proses evaluasi dan eksperimen model dapat dilihat pada file ProjectDL.ipynb, yang berisi tahapan pelatihan, validasi, serta pengujian performa model. Metode ini memungkinkan analisis terhadap akurasi dan kesalahan klasifikasi pada masing-masing motif batik. Dengan adanya pemisahan data train dan test yang jelas, proyek ini telah menerapkan praktik machine learning yang baik untuk menghindari overfitting dan memastikan reliabilitas hasil.

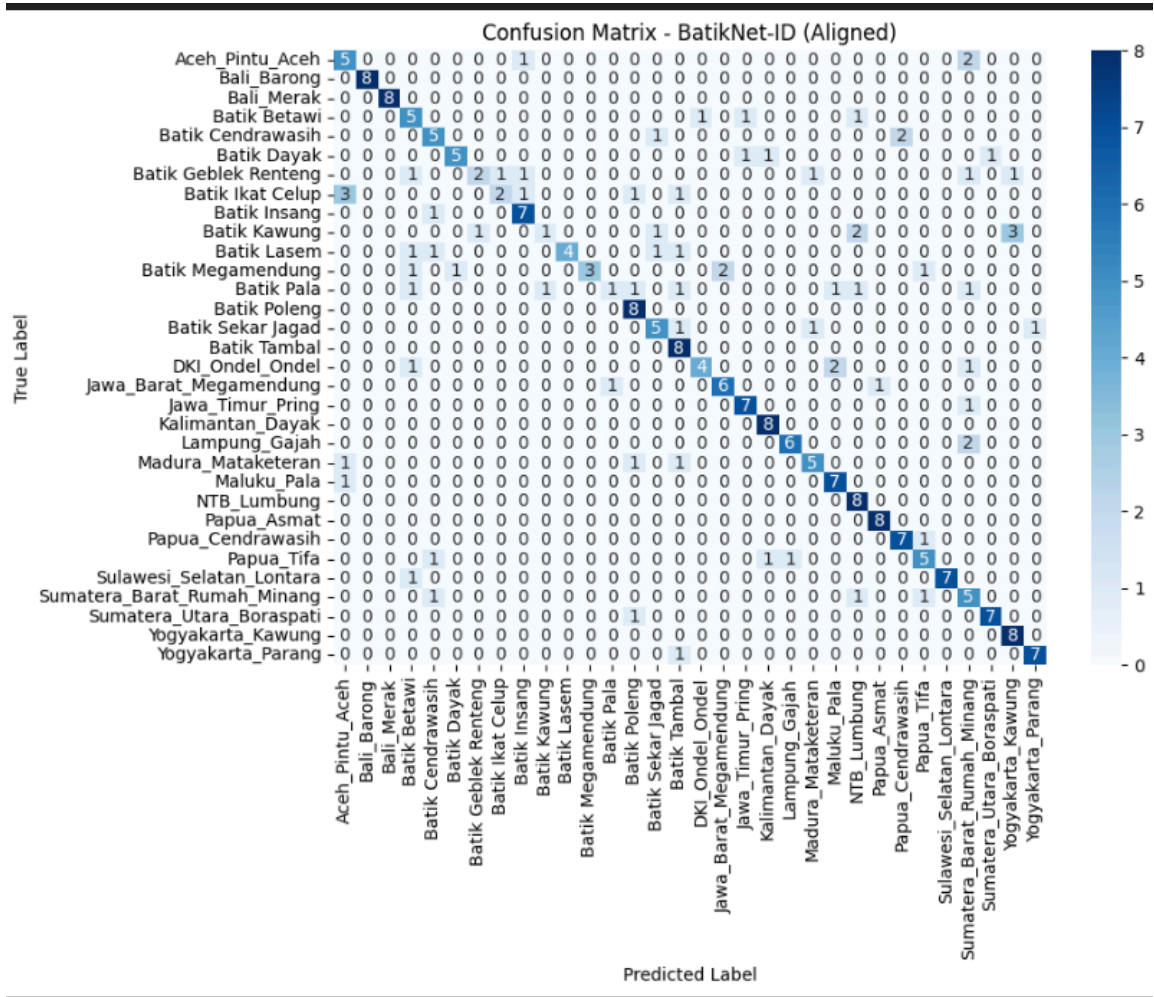
Model Evaluation :

```
loss, accuracy = model.evaluate(val_ds, verbose=0)

print(f"Validation Loss      : {loss:.4f}")
print(f"Validation Accuracy : {accuracy:.4f}")

Validation Loss      : 1.1720
Validation Accuracy : 0.7109
```

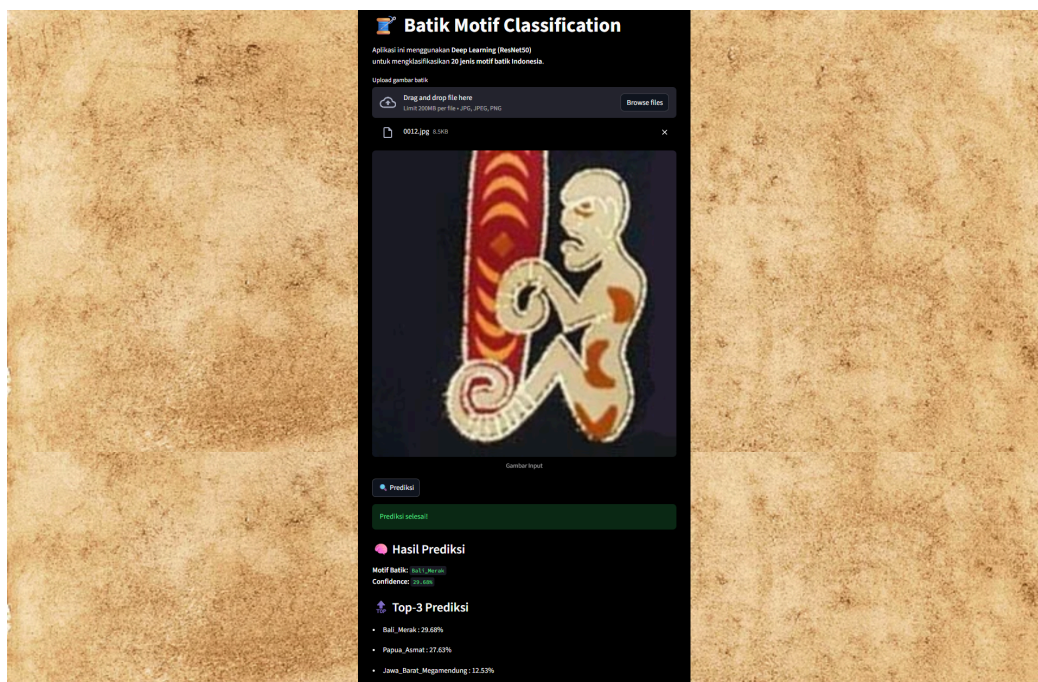
Confusion Matrix :



Classification Report :

Classification Report:				
	precision	recall	f1-score	support
Aceh_Pintu_Aceh	0.5000	0.6250	0.5556	8
Bali_Barong	1.0000	1.0000	1.0000	8
Bali_Merak	1.0000	1.0000	1.0000	8
Batik_Betawi	0.4545	0.6250	0.5263	8
Batik_Cendrawasih	0.5556	0.6250	0.5882	8
Batik_Dayak	0.8333	0.6250	0.7143	8
Batik_Geblek_Renteng	0.6667	0.2500	0.3636	8
Batik_Ikat_Celup	0.6667	0.2500	0.3636	8
Batik_Insang	0.7000	0.8750	0.7778	8
Batik_Kawung	0.5000	0.1250	0.2000	8
Batik_Lasem	1.0000	0.5000	0.6667	8
Batik_Megamendung	1.0000	0.3750	0.5455	8
Batik_Pala	0.5000	0.1250	0.2000	8
Batik_Poleng	0.6667	1.0000	0.8000	8
Batik_Sekar_Jagad	0.6250	0.6250	0.6250	8
Batik_Tambal	0.5714	1.0000	0.7273	8
DKI_Ondel_Ondel	0.8000	0.5000	0.6154	8
Jawa_Barat_Megamendung	0.7500	0.7500	0.7500	8
Jawa_Timur_Pring	0.7778	0.8750	0.8235	8
Kalimantan_Dayak	0.8000	1.0000	0.8889	8
Lampung_Gajah	0.8571	0.7500	0.8000	8
...				
accuracy			0.7109	256
macro avg	0.7296	0.7109	0.6902	256
weighted avg	0.7296	0.7109	0.6902	256

V. Deployment



Link Deployment : <https://website-batik-classification.streamlit.app/>

Link Github: <https://github.com/aidennz/Batik-Classification.git>

VI. Reflection

Secara keseluruhan, proyek ini telah mengintegrasikan seluruh tahapan *deep learning pipeline* dengan baik, mulai dari pengelolaan dataset, pelatihan model, evaluasi, hingga deployment. Penggunaan ResNet50 dengan fine-tuning menunjukkan pemahaman yang baik terhadap pemanfaatan transfer learning untuk permasalahan klasifikasi citra. Selain itu, deployment menggunakan Streamlit memberikan nilai tambah karena membuat sistem lebih aplikatif dan mudah diakses. Ke depan, proyek ini masih dapat dikembangkan lebih lanjut, misalnya dengan menambah jumlah dan variasi data batik, meningkatkan evaluasi menggunakan metrik lain seperti confusion matrix, atau mengoptimalkan tampilan dan fitur aplikasi agar lebih informatif bagi pengguna.

VII. Reference

- <https://www.kaggle.com/datasets/hendryhb/batik-nusantara-batik-indonesia-dataset>
- Meranggi, D. G. T., Yudistira, N., & Sari, Y. A. (2021). *Batik classification using convolutional neural network with data improvements*. *Journal of Open Innovation: Technology, Market, and Complexity*.
<https://joiv.org/index.php/joiv/article/view/716/402>
- Rangkuti, A. H., Harjoko, A., & Putra, A. (2021). *A novel reliable approach for image batik classification that invariant with scale and rotation using MU2ECS-LBP algorithm*. *Procedia Computer Science*, 179, 214–221.
<https://www.sciencedirect.com/science/article/pii/S1877050921001058>